总思路：通过前三题分别研究启发函数、路径计费和剪枝顺序，再在综合题中将这些方法两两结合进行对比，按照不同的标准给出最佳方案

A*、Alpha-beta及宝藏博弈的实验分析

姓名：____张云飞_____ 学号：_____10245501481_____ github：https://github.com/yun12345-oss/artificial-intelligence.git

题目一：迷宫寻路

目标与设计：对照启发函数如何改变扩展次数和单次查询总耗时，并结合数据分析原因，得到每个启发函数的优劣

表1a A*概念	如何计算或操作	在搜索中的作用
状态 n=(r,c)	用行、列表示当前位置，向上下左右的可通行邻格移动。	从当前位置生成合法后继。
g(n) 已走代价	起点 g=0。走到邻格 v 时，候选 g(v)=g(u)+1。	保存已找到的最短到达代价。发现更小 g 时更新父节点https://github.com/yun12345-oss/artificial-intelligence.git并重新入队。
h(n) 剩余估计	零启发取0。曼哈顿取$\|r-r_E\|+\|c-c_E\|$。BFS在原障碍图上从终点反向求最少剩余步数。	三种 h 都是到终点的剩余步数下界。
f(n) 节点评价	f(n)=g(n)+h(n)。	f 越小，待搜索节点的优先级越高。
OPEN 优先队列	最小堆先取 f 最小的记录。同 f 按入队先后处理，过期记录出队时跳过。	终点有效出队后，沿父节点回溯路径。

探究一 最短路径形成与曼哈顿的队列排序

零启发 7步 / 扩展15次　　曼哈顿 7步 / 扩展13次　　BFS下界 7步 / 扩展7次

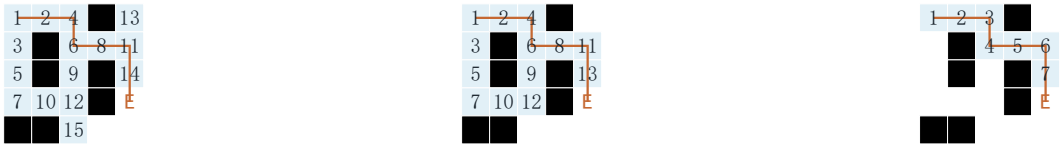


图1a – c 数字是实际扩展序号，橙线是返回路径。起点 S=(0,0)，终点 E=(3,4)。

表1b 返回路径、父节点与扩展序号

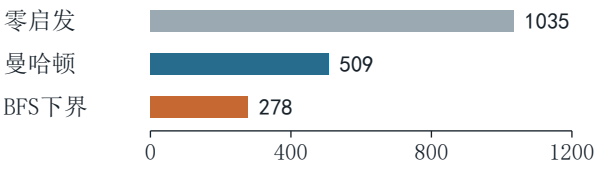
g/步	状态	父节点	h=0	曼哈顿	BFS
0	(0,0)	—	1	1	1
1	(0,1)	(0,0)	2	2	2
2	(0,2)	(0,1)	4	4	3
3	(1,2)	(0,2)	6	6	4
4	(1,3)	(1,2)	8	8	5
5	(1,4)	(1,3)	11	11	6
6	(2,4)	(1,4)	14	13	7
7	(3,4)	(2,4)	15	13	7

表1c 曼哈顿 OPEN 队列，节点记为坐标[g,h,f]

状态	A* 的状态、g/h/f 与优先队列的变化依据（取关键三步）
(0,1)	(1,0) [1,6,7]→(0,2) [2,5,7] 两个节点的 f 都是7，先入队的(1,0)排在前面。
(1,4)	(3,2) [5,2,7]→(2,4) [6,1,7]→(0,4) [6,3,9] 两个新邻格的 g=5+1=6。下方(2,4)的 h=1，上方(0,4)的 h=3。原有的(3,2)与(2,4)同为 f=7，且更早入队。
(2,4)	(3,4) [7,0,7]→(0,4) [6,3,9]→(4,2) [6,3,9] 终点的 g=6+1=7，h=0，f=7，小于两个旁支的9。终点先出队，回溯返回路径。

探究二 绕行图中三种启发函数的扩展差异

图1d 绕行图的搜索扩展次数

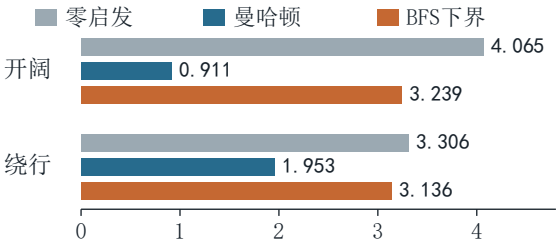


三种方法在绕行图上都得到43步的最短路径。**零启发**的 f=g，只按从起点走了多少步来排队。只要到某个节点的最短距离小于43步，搜索就会在终点之前扩展它，即使这个节点位于远离终点的旁支上。因此，零启发共扩展1,035次。

曼哈顿距离把预计剩余步数 h 加入优先级。走一步若在几何上靠近终点，g 加1、h减1，f保持不变。若远离终点，g和h 各加1，f上升2，节点就可能排到后面。这使扩展降至509次，减少约51%。但起点的曼哈顿距离只有33步，实际需要43步，说明墙壁造成的绕行仍会让部分看似接近终点的旁支排在前面。**BFS下界**把障碍计入剩余步数，进一步将扩展降至278次。

探究三 三种启发函数在不同地图上的耗时比较

图1e 单次查询总耗时中位数（ms）



开阔图中，**零启发**只按已走步数向四周扩展，因此处理了许多偏离终点的节点，共扩展1,259次。**曼哈顿距离**在开阔图上等于真实剩余步数，使扩展降至216次。**BFS下界**给出的剩余步数相同，扩展数也是216次，但反向建表平均耗时2.339 ms。这笔额外开销解释了为什么两者搜索量相同，BFS的总耗时却明显高于曼哈顿。

绕行图中，**曼哈顿**仍能引导搜索朝终点推进，扩展数从零启发的1,035次降至509次。由于曼哈顿距离忽略墙壁，部分看似接近终点的节点实际需要绕远，仍会被提前扩展。**BFS**把绕行计入剩余步数，进一步将扩展降至278次，比曼哈顿平均节省约0.821 ms的搜索时间。但一次建表平均需要2.096 ms，超过了节省的搜索时间，所以单次查询中曼哈顿最快，BFS的总耗时与零启发接近。

两图均为同组60×60地图的首次查询。柱长为总耗时中位数，分段均值用于解释准备和搜索开销。

表1d 方法	优势	代价与适用条件
零启发	无需估值和建表，作为统一A*基线。	无目标引导，本例扩展最多，可用于基线核验。
曼哈顿	估值简单，在四方向单位代价开阔图中精确。	忽略绕行，本例单次查询总耗时较低。
BFS下界	单位代价下精确，识别绕行，固定终点可复用。	需建表和存储，换终点通常需重建。

题目二：加权地形

目标与设计：对照在免费终点下，证明修正的启发函数的可采纳性，展示最低路费路径的形成与三种启发函数的扩展差异（中间会添加自设地形），并结合数据分析原因，得到每个启发函数的优劣。

表2a 方法	免费终点下的建模	信息与准备成本
修正曼哈顿	$h_M=m\ \max(D_1-1,0)$	几何距离与最小进入代价，无需建表
修正BFS下界	$h_B=m\ \max(L-1,0)$	保留障碍的步数L，需一次BFS
有向地标	$d(a,E)-d(a,n)$；$d(n,a)-d(E,a)$ 两类差与0在全部地标a上取最大。	4个固定地标，正反共8次Dijkstra

探究一 三种启发函数的可采纳性证明与实例核验

d*为真实余费，m不含E；邻E时普通曼哈顿为1、真实费用为0，因此需要适配免费终点。

修正曼哈顿
至少移动D_1步，末步免费，其余每步≥m，因此：
$h_M=m\ \max(D_1-1,0)\ \leqslant\ d^*$

修正BFS下界
障碍下至少移动L步，末步免费，其余每步≥m，因此：
$h_B=m\ \max(L-1,0)\ \leqslant\ d^*$

有向地标
由免费图上的三角不等式：
$d(a,E)-d(a,n)\ \leqslant\ d^*$
$d(n,a)-d(E,a)\ \leqslant\ d^*$

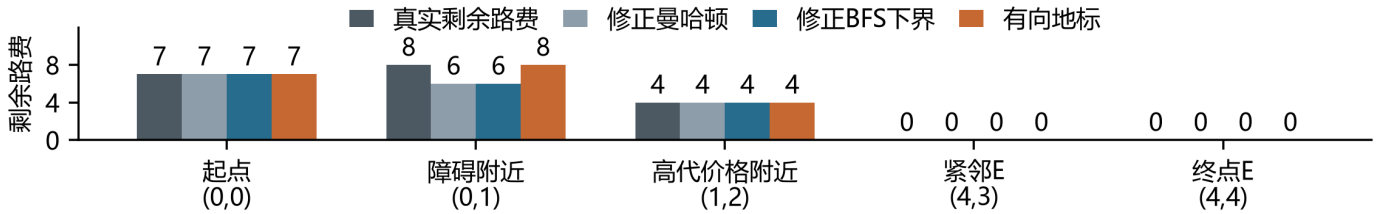


图2a，(0,1)真实路费8，修正曼哈顿和修正BFS下界为6，有向地标为8；邻E及E均为0。三种估计均不超过真实值

探究二 最低路费路径形成与三种启发函数的扩展差异

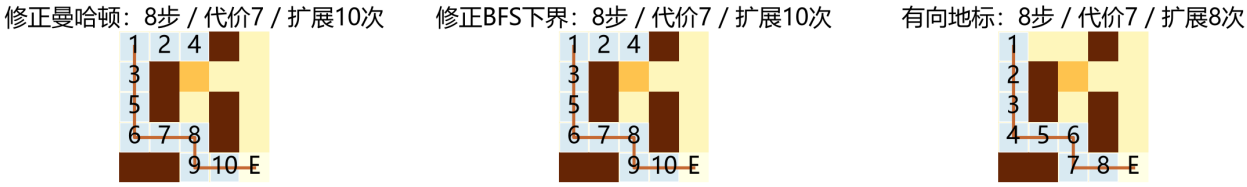


图2b—d：蓝格数字为扩展序号，橙线为路径，浅棕格可通行；坐标(行,列)，S=(0,0)，E=(4,4)。

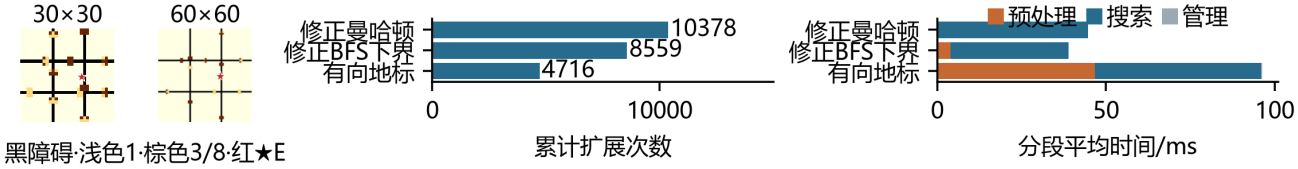
步序	状态	入费	累计g	修正曼哈顿	修正BFS下界	有向地标
0	(0,0)	0	0	1	1	1
1	(1,0)	1	1	3	3	2
2	(2,0)	1	2	5	5	3
3	(3,0)	1	3	6	6	4
4	(3,1)	1	4	7	7	5
5	(3,2)	1	5	8	8	6
6	(4,2)	1	6	9	9	7
7	(4,3)	1	7	10	10	8
8	(4,4)	0	7	10	10	8

修正曼哈顿与**修正BFS下界**均扩展10次。 官方图没有障碍，BFS算出的最少剩余步数等于曼哈顿距离，两者扣除免费进入终点的最后一步后，得到相同的启发值。它们按最小进入代价估计剩余路费，容易低估高权格带来的开销。例如，旁支 (0,1)的实际剩余路费为8，两种方法都估为6，因此这个旁支较早出队。到扩展 (2,0) 时，两种方法都已额外扩展 (0,1)和 (0,2)，累计5次。

有向地标只扩展8次。 地标下界使用加权距离，在 (0,1) 给出的剩余路费为8，使该旁支的 f=g+h 更高。有向地标因此先推进低路费路径，到扩展 (2,0) 时只累计3次，终点出队前也没有扩展上述两个旁支。三种方法最终都得到最小路费7，扩展数的差异来自对旁支剩余路费的估计。

探究三 自设加权房间图：探究搜索差异、时间成本与适用条件

固定终点前1/5/10/20次查询，冷启动、任务内复用；图示种子20261111，性能为60×60、20次。



橙：建表预处理；蓝：A*搜索；灰：管理。分段为均值，总时间为中位数

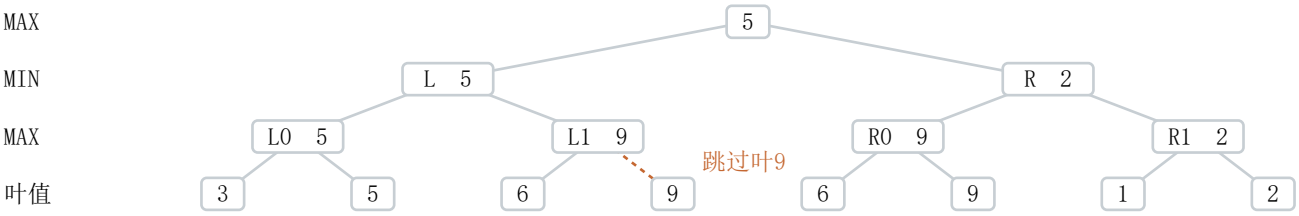
表2e	修正曼哈顿	修正BFS下界	有向地标
扩展数差异	10378次；几何下界忽略绕行和高权值。	8559次，少1819次；障碍步数减少绕行搜索。	4716次，少5662次；加权距离约束改善引导。
时间差异	43.720 ms；无建表成本，但搜索较多。	38.834 ms；复用使搜索节省覆盖准备成本。	96.691 ms；8次预处理及估值成本未抵消。

总结与适用条件：**修正曼哈顿**免建表，宜用于准备预算低、查询少或终点常变的任务；
修正BFS下界适合固定终点、绕行明显的重复查询。
有向地标可在加权约束强、准备可充分复用时评估，但本轮未获时间优势，改变免费终点需重查复用。

3 博弈回传、剪枝与访问顺序

目标与设计：以MiniMax为基线，自设树比较访问顺序对 Alpha-Beta 剪枝和耗时的影响，并结合数据分析原因，，再放到官方树进行验证，最后得到每个访问顺序的优劣。

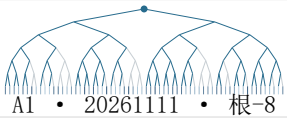
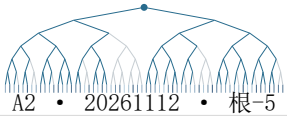
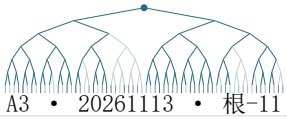
探究一 官方小树的数值回传与剪枝依据



图中为精确回传值**已经记录MiniMax过程， Alpha-Beta 剪枝**中α是MAX已确保的下界， β是MIN已确保的上界。原始编号下，L0回传5，L令β=5。L1读到6即满足α ≥ β，跳过叶9，返回下界6，精确值为9。

探究二 生成树 A（二叉、深度6）的五顺序对照

表内均为 Alpha-Beta 的访问数 / 总耗时（μs，含排序准备）。树图蓝边为原始编号已访问边，灰边未访问。**理想序**按精确子树值在MAX层降序、MIN层升序，**理想逆序**反向（**分别作为最少访问节点和最多访问节点数**）。**估值序**以叶值均值替代精确值。

表3a 实例树	理想顺序	原始编号	估值顺序	固定随机顺序	理想逆序	Minimax 基线
 A1 • 20261111 • 根-8	44 / 433.97	91 / 278.86	67 / 500.47	80 / 239.67	126 / 633.88	127 / 276.97
 A2 • 20261112 • 根-5	44 / 497.17	87 / 265.46	54 / 472.40	98 / 292.72	122 / 614.47	127 / 279.64
 A3 • 20261113 • 根-11	44 / 446.59	96 / 288.55	45 / 476.35	85 / 265.14	127 / 650.08	127 / 281.53

节点差异 **理想顺序**在 MAX 层先访问高值分支、MIN 层先访问低值分支，使α β边界较早收紧，三个实例都只访问44个节点。**理想逆序**把有利分支放在后面，搜索难以及早排除其余分支，因此访问126、122、127个节点，接近 Minimax 的完整遍历。**估值顺序**用子树叶值均值引导搜索，比编号序分别少访问24、33、51个节点。但均值会受其他叶值影响，与决定回传结果的最大值或最小值排序存在差异，因此三棵树的剪枝收益不同。**固定随机顺序**没有价值引导，恰好提前访问有利分支时就能减少搜索，所以在 A1、A3 上各少访问11个节点，在 A2 上却多访问11个。

时间差异 **估值顺序**需要预先遍历树、计算叶值均值并排序。三棵树虽然分别节省约86、104、146 μs的搜索时间，准备工作却需要约307 – 335 μs，因此总耗时均高于编号序。**两种理想顺序**同样需要预求精确子树值，准备成本使理想顺序即使访问最少，总耗时仍高于 Minimax。**固定随机与编号序**的准备开销接近，时间差异因而主要随搜索量变化，A1、A3更快，A2更慢。编号 Alpha-Beta 虽比 Minimax 少访问节点，但还要维护和检查α β 边界，在这组127节点的小树上，节省的访问量仅使 A2 的总耗时低于 Minimax。

探究三 官方小树的五顺序复核

沿用上图编号。五种顺序的根值均为5。叶子列计实际评价次数。计时含顺序准备，轨迹另行记录。

Minimax 基线（原始编号）
15/8 37.08 μs（独立批次）

表3b 顺序	访问与剪枝过程	节点/叶子	总耗时/μs
理想顺序	L0 读5、3，L将β降至5。L1先读9， α=9≥β，跳过6。 L回传5，根的α升至5。R1先读2、1，R的β降至2≤α，跳过R0。	11 / 5	63.70
原始编号	**L0 读3、5，L的β降至5。L1先读6， α=6≥β，跳过9。 根的α升至5。R0读6、9， β=9>α。R1读1、2后β=2≤α，已无分支可剪。**	**14 / 7**	**41.49**
估值顺序	叶均值排出与理想序相同的次序。L0读5、3，L1读9后α=9≥β=5，跳过6。 根的α升至5，R1读2、1后β=2≤α，跳过R0。	11 / 5	64.32
固定随机顺序	先读L1的9、6，L的β降至9。L0读5、3，未达到β=9。 根的α升至5。R1读2、1后，R的β=2≤α，跳过R0。	12 / 6	35.72
理想逆序	先走R。R0读6、9， β=9；R1读1、2，根的α仅升至2。 再走L，L1读6、9， β=9。L0读3、5，未触及β。八个叶子全部访问。	15 / 8	74.74

复核结果 理想序先建立有效边界，访问11个节点；逆序访问15个。固定随机序先看R1，跳过R0，比编号序少访2个，本例总耗时也最低。估值序虽比编号序少访3个，搜索约省9 μs，准备仍约需32 μs，总耗时由41.49升至64.32 μs。这棵树重现了探究二的边界与准备成本现象。

原始编号、估值顺序与固定随机顺序的优劣

原始编号	估值顺序	固定随机顺序
优势 直接沿用输入顺序，准备开销低，适合作为对照基线。	**优势** 用叶值均值提供价值引导，有机会优先访问有利分支，更早建立剪枝边界。	**优势** 准备成本低，固定排列可复现，便于检验输入编号对搜索的影响。
劣势 效果依赖输入编号。若有利分支排在后面，搜索可能在边界收紧前访问较多节点。	**劣势** 需要计算估值并排序。均值可能与真实回传值的排序不一致，节省的搜索也可能不足以覆盖准备成本。	**劣势** 排列缺少价值引导，可能提前访问有利分支，也可能将其推后。剪枝收益随实例与固定排列而变化。

4 宝藏博弈的深度决策与组合对照

目标与设计：观察搜索深度怎样改变首步，并比较**预建距离表**和**实时测量**时不同寻路启发函数与不同博弈策略的**组合**的总体开销，得到**不同标准下**的最优组合，并结合数据分析原因。

探究一 距离矩阵、状态表示与首步变化

图4a 官方地图 01

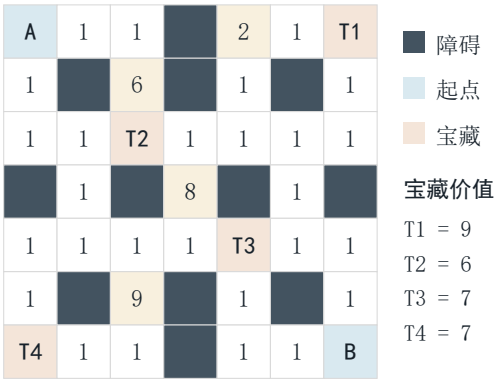


表4a 六个关键位置的最小路费

起点/终点	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

格内数字是进入代价，宝藏格进入代价为1。矩阵对角线为0，其他距离由 A* 求出

$s = (p_A, p_B, R, S_A, S_B, \text{当前玩家})$， 单步净收益 $= V(T_i) - d(p, T_i)$， $U = S_A - S_B$

R 为剩余宝藏集合。玩家选取宝藏后查表扣除路费，更新自身位置与累计得分，移除目标并切换玩家。即使净收益为负也要行动。经过其他宝藏不会自动收集，另一玩家不阻路。

图4b 四种首步的精确评价

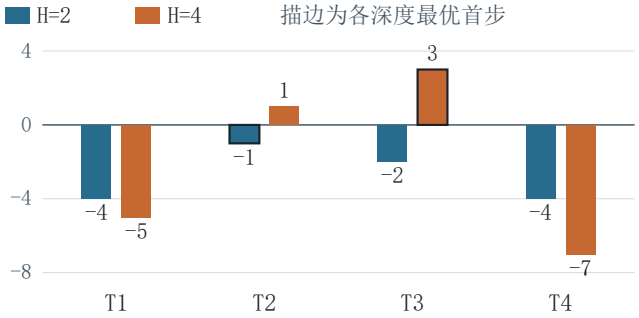


表4b 最优应对与逐步净收益

深度	最优应对序列	A、B累计	得分差
H=2	A取T2(+2) → B取T3(+3)	2、3	-1
H=4	A取T3(-1) → B取T1(+1) → A取T2(0) → B取T4(-5)	-1、-4	3

首步变化 H 按单次行动计。H=2 时，T2 的两步评价为 -1，高于 T3 的 -2。H=4 覆盖完整对局，T3 首步虽亏1点，但沿表中应对，B 最后从 T1 到 T4 花12点路费、只得7点价值。双方累计为 -1 和 -4，T3 的完整评价升至3，高于 T2 的1。后续位置和剩余宝藏改变了双方的移动成本。

探究二 预建距离表时的 12 种组合

每个深度均比较三种寻路启发函数和四种博弈策略。各组合先用 A* 求出30个非对角有向距离，再查表搜索。总耗时包含启发函数准备、建表和博弈搜索，取 01 同轮测量的9批中位数。

H=2

A* 下界	编号 Minimax	编号 Alpha-Beta	**估值**排序 Alpha-Beta	固定随机顺序 Alpha-Beta
曼哈顿下界	1.629 17 节点	1.781 13 节点	1.644 11 节点	1.859 15 节点
BFS步数下界	1.701 17 节点	1.760 13 节点	1.751 11 节点	1.950 15 节点
有向地标下界	3.429 17 节点	3.419 13 节点	3.553 11 节点	3.717 15 节点

H=4

A* 下界	编号 Minimax	编号 Alpha-Beta	**估值**排序 Alpha-Beta	固定随机顺序 Alpha-Beta
曼哈顿下界	1.975 65 节点	1.860 47 节点	1.878 37 节点	2.334 35 节点
BFS步数下界	2.003 65 节点	2.011 47 节点	1.951 37 节点	2.518 35 节点
有向地标下界	3.997 65 节点	3.906 47 节点	3.548 37 节点	3.945 35 节点

每格上行为总耗时中位数（ms），下行为博弈访问节点数。两表共用下方色标。

总耗时（ms） 1.6 4.1

同一行的差异来自剪枝收益在总任务中所占的比例。 各组合都先完成30次寻路建表，剪枝主要减少后续博弈搜索。H=2时，**曼哈顿行**的收益排序将访问量从 Minimax 的17个节点降至11个，总耗时却接近，因为浅层博弈本身开销较小。H=4时，博弈访问量增加，**编号 Alpha-Beta** 将65个节点减至47个，总耗时从1.975降至1.860 ms。估值排序进一步减至37个节点，但总耗时为1.878 ms，与编号序的四分位范围重叠。博弈阶段省下的时间较少，容易被占比较大的建表耗时及其波动掩盖。

同一列的差异主要来自获取启发信息的成本。 三种启发函数最终得到相同的最小路费矩阵，因此固定博弈策略后，访问节点数相同。以 H=4**估值排序**为例，**BFS和地标**将 A* 扩展数从329降至304和287，但**BFS**需要额外构建终点距离表，**地标**需要8次Dijkstra预处理，搜索时还要计算多个距离差。本例地图较小，减少这些扩展所节省的时间不足以覆盖额外开销，因此总耗时从**曼哈顿**的1.878 ms升至**BFS**的1.951 ms和**地标**的3.548 ms。

探究三 实时寻路时的 12 种组合

沿用第4页的官方样例 01、行列顺序及深度设置。实时模式按需调用 A* 求路费。总耗时包含启发函数准备、实际寻路与博弈搜索，取同轮测量的9批中位数。

H=2

A* 下界	编号 Minimax	编号 Alpha-Beta	**估值**排序 Alpha-Beta	固定随机顺序 Alpha-Beta
曼哈顿下界	0.959 17 / 16	0.677 13 / 12	0.863 11 / 16	0.958 15 / 14
BFS步数下界	0.939 17 / 16	0.728 13 / 12	0.984 11 / 16	1.036 15 / 14
有向地标下界	1.988 17 / 16	1.607 13 / 12	2.089 11 / 16	2.036 15 / 14

H=4

A* 下界	编号 Minimax	编号 Alpha-Beta	**估值**排序 Alpha-Beta	固定随机顺序 Alpha-Beta
曼哈顿下界	3.540 65 / 64	2.518 47 / 46	2.541 37 / 44	2.595 35 / 34
BFS步数下界	3.551 65 / 64	2.605 47 / 46	2.524 37 / 44	2.612 35 / 34
有向地标下界	7.523 65 / 64	5.350 47 / 46	5.122 37 / 44	4.486 35 / 34

每格上行为总耗时中位数（ms），下行为博弈节点数 / A* 调用次数。两表共用下方色标。

总耗时（ms） 0.6 7.6

同一行比较博弈策略。 在 H=2的曼哈顿行，编号 Alpha-Beta 将 Minimax 的博弈访问量从17个节点降至13个，A调用也从16次降至12次，总耗时从0.959降至0.677 ms。估值排序进一步把访问量降至11个节点，A调用却回升至16次，总耗时也升至0.863 ms。原因是排序需要先计算候选行动的路费，有些分支虽然随后被剪掉，寻路开销已经发生。实时寻路的耗时因此取决于剪枝省掉了多少次寻路，以及排序提前触发了多少次寻路。

同一列比较寻路启发函数。 固定 H=4和编号 Alpha-Beta，三种方法都访问47个博弈节点、调用46次A*，因此差异主要来自每次寻路及其准备开销。曼哈顿、BFS和有向地标的A*累计扩展数分别为458、416、399，总耗时却依次为2.518、2.605、5.350 ms。BFS需要为目标构建步数表，有向地标需要预先计算加权距离，搜索时还要计算多个距离差。本例中，减少扩展节省的时间不足以抵消这些开销，因此曼哈顿的扩展数最多，总耗时最低。

题目四总结 按评价标准选方案

表5a 汇总第4、5页的官方 01 数据。不同标准分别排名，时间项采用总耗时中位数。

评价标准	H=2	H=4
端到端耗时最低	实时寻路＋曼哈顿＋编号 Alpha-Beta **0.677 ms**	预建表＋曼哈顿＋编号 Alpha-Beta **1.860 ms**
A* 调用次数最少	实时寻路＋编号 Alpha-Beta 12次，三种启发函数并列	预建表 30次，各组合并列
A* 扩展节点最少	实时寻路＋有向地标＋编号 Alpha-Beta 111次扩展	预建表＋有向地标 287次扩展，四种博弈策略并列
博弈访问节点最少	**估值**排序 Alpha-Beta 11个节点，两种模式、三种下界并列	固定随机顺序 Alpha-Beta 35个节点，两种模式、三种下界并列

整体总结 从寻路到博弈选型

题目	对综合题的贡献
题目一 最短路径	以独立 BFS 核对 A* 最短路，并比较启发信息、扩展量与准备成本。题目四沿用这套方法求关键位置间的距离。
题目二 最小路费	按进入格累计路费，依据计费规则构造下界。题目四沿用加权状态转移，并用“宝藏价值减路费”计算行动净收益。
题目三 回传与剪枝	以 Minimax 核对 Alpha-Beta 的根值，区分剪枝节省的访问量与排序开销。题目四据此比较四种博弈策略。
题目四 联合选型	将 A* 路费接入博弈状态，把寻路、建表与剪枝放在同一计时范围内。两步搜索时（实时寻路＋曼哈顿＋编号 Alpha-Beta ）更省时，四步完整搜索时（预建表＋曼哈顿＋编号 Alpha-Beta ）预建表更省时。